

Part01 – Questions & Answers

Q1: What is the purpose of the finally block?

A: The finally block contains code that always runs after the try/catch, whether an exception was thrown or not, and whether or not it was caught. It's used for cleanup actions (closing files, releasing resources, printing a completion message) that must happen regardless of the outcome of the operation.

Q2: How does `int.TryParse()` improve program robustness compared to `int.Parse()`?

A: `int.Parse()` throws a `FormatException` (or `ArgumentNullException/OverflowException`) if the input is invalid, which crashes the program unless it's wrapped in a try/catch. `int.TryParse()` instead returns a boolean indicating success or failure and outputs the parsed value through an out parameter, so invalid input can be handled gracefully without relying on exception handling.

Q3: What exception occurs when trying to access Value on a null `Nullable<T>`?

A: Accessing the Value property of a `Nullable<T>` that has no value (`HasValue == false`) throws an `InvalidOperationException`.

Q4: Why is it necessary to check array bounds before accessing elements?

A: Arrays in C# have a fixed size, and accessing an index outside the valid range (less than 0 or greater than or equal to the array's Length) throws an `IndexOutOfRangeException` at runtime. Checking bounds first prevents the program from crashing and allows it to handle invalid indices safely.

Q5: How is the `GetLength(dimension)` method used in multi-dimensional arrays?

A: `GetLength(dimension)` returns the number of elements in a specific dimension of a multi-dimensional array, where dimension is a zero-based index (0 for rows, 1 for columns, and so on). It's commonly used to control loop bounds when iterating over rows and columns instead of hardcoding the size.

Q6: How does the memory allocation differ between jagged arrays and rectangular arrays?

A: A rectangular array (e.g. `int[,]`) is allocated as a single contiguous block of memory with a fixed number of rows and columns, so every row has the same length. A jagged array (e.g. `int[][]`) is an array of references to separate arrays, each allocated independently, so each row can have a different length and the memory is not contiguous across rows.

Q7: What is the purpose of nullable reference types in C#?

A: Nullable reference types (enabled with `#nullable enable`) let the compiler track and warn about reference variables that might be null, helping catch potential `NullReferenceExceptions` at compile time instead of at runtime. Types are treated as non-nullable by default unless explicitly declared nullable with a `?` (e.g. `string?`).

Q8: What is the performance impact of boxing and unboxing in C#?

A: Boxing wraps a value type in an object on the heap, and unboxing extracts it back into a value type. Both operations involve extra memory allocation and copying that plain value-type operations don't require, which adds overhead and increases garbage collection pressure. Frequent boxing/unboxing, especially in loops or collections of value types, can noticeably slow down a program compared to using generics or working directly with value types.

Q9: Why must out parameters be initialized inside the method?

A: An out parameter is not required to have a value before the method call, so the compiler enforces that the called method must assign a value to it before the method returns. This guarantees the caller always receives a valid, definitely-assigned value once the method completes.

Q10: Why must optional parameters always appear at the end of a method's parameter list?

A: Optional parameters must come after all required parameters so the compiler can match arguments positionally without ambiguity. If an optional parameter appeared before a required one, the compiler wouldn't be able to tell whether an omitted argument belongs to the optional parameter or shifts the following arguments, so C# does not allow that ordering.

Q11: How does the null propagation operator prevent `NullReferenceException`?

A: The null-conditional (propagation) operator (`?.`) checks whether the object on its left is null before accessing a member or calling a method. If the object is null, the expression short-circuits and returns null instead of throwing a `NullReferenceException`, avoiding the need for explicit null checks.

Q12: When is a switch expression preferred over a traditional if statement?

A: A switch expression is preferred when a variable is being compared against several discrete values or patterns and each case simply produces a value to assign or return. It is more concise and readable than a long if/else-if chain, and the compiler can check for exhaustiveness. Traditional if statements are better suited for complex conditions involving ranges, multiple variables, or logical combinations.

Q13: What are the limitations of the `params` keyword in method definitions?

A: A method can have only one `params` parameter, and it must be the last parameter in the parameter list. The `params` parameter must be a single-dimensional array type. Passing many arguments through `params` can also incur a small performance cost since an array is allocated to hold them, and only one `params` array is allowed per method signature.

Self study

My Own Exception

Custom Exceptions

Definition

A **Custom Exception** is an exception created by the developer to handle a **specific business rule or business case** in the application.

Why Do We Use Custom Exceptions?

- To represent a **specific business error**.
- To make error messages more **meaningful and clear**.
- To handle specific errors separately.
- To make the code more **organized and readable**.

When Do We Use It?

We use a Custom Exception when the problem is related to a **business rule**, rather than a standard programming error.

Example:

If a system has a rule that a student must be at least 18 years old:

```
if (age < 18)
{
    throw new InvalidAgeException("Age must be 18 or older.");
}
```

}

Here, InvalidAgeException can be a Custom Exception created specifically for this business rule.

Important Note

We don't need to create a Custom Exception for every error.

For common errors, we can use the built-in exceptions:

DivideByZeroException

IndexOutOfRangeException

NullReferenceException

FormatException

But when we have a **specific business requirement**, creating a Custom Exception can make the error more meaningful.

Key Idea

Custom Exception → Specific Business Rule → Meaningful Error → Specific Handling

Inheritance Design Concept

Casting a base (parent) class reference to a derived (child) class reference is known as **Downcasting**.

Here is the breakdown for your self-study notes on how this relates to object-oriented design and how to handle it correctly in C#.

Relation to Inheritance Design Concepts

- **The "Is-A" Relationship:** Inheritance dictates that a derived class *is a* base class. For example, if Manager inherits from Employee, every Manager is an Employee. Because of this, **Upcasting** (child to parent) is always 100% safe and implicit. However, **Downcasting** reverses this. You are asking the compiler to treat a general Employee as a specific Manager. This is dangerous because the Employee in memory might actually be a Developer or a Janitor.
- **Polymorphism:** Downcasting is often a symptom of trying to bypass polymorphism. In a perfectly designed polymorphic system, the parent class defines a virtual method (e.g., Work()), and you rely on the child classes to override it. If you find yourself constantly downcasting to access child-specific methods, it may indicate a flaw in your class hierarchy design.
- **Liskov Substitution Principle (LSP):** This SOLID principle states that objects of a superclass should be replaceable with objects of its subclasses without breaking the application. Forcing a downcast often risks violating this principle by assuming a specific derived type rather than working with the abstraction.

How to Safely Downcast in C# Because downcasting is inherently unsafe, attempting a direct cast (e.g., Manager m = (Manager)myEmployee;) will crash your application with an InvalidCastException if the object is not actually a Manager.

Modern C# provides two safe ways to handle this:

1. The is Operator (Pattern Matching - Recommended) This checks the type and safely assigns it to a new variable in a single, clean step.

C#

```
Employee myEmployee = new Manager();
```

```
// If myEmployee is a Manager, it safely casts it and assigns it to 'm'
```

```
if (myEmployee is Manager m)
```

```
{
```

```
    m.ApproveTimesheets(); // 'm' is safely a Manager here
```

```
}
```

2. The as Operator This attempts the cast and returns null if it fails, rather than throwing an exception. You must check for null afterward.

```
C#
```

```
Employee myEmployee = new Developer();
```

```
Manager m = myEmployee as Manager;
```

```
if (m != null)
```

```
{
```

```
    m.ApproveTimesheets();
```

```
}
```

```
//// 4- Unmanaged resources recovering >> self
```

In C#, the Garbage Collector (GC) is highly efficient at automatically cleaning up standard objects in memory (managed resources). However, it does not know how to handle **unmanaged resources**—these are native operating system objects like open file handles, active database connections, network sockets, or graphic window handles.

If you do not explicitly recover (release) unmanaged resources, your application will leak memory, and the operating system may keep files locked indefinitely.

Here is how unmanaged resources are recovered in C#:

1. The IDisposable Interface (The Primary Method) The foundational way to handle unmanaged resources is by implementing the IDisposable interface. This forces your class to provide a Dispose() method. The developer is expected to manually call .Dispose() when they are finished with the object, which contains the exact code to close the file or sever the database connection.

2. The using Statement (The Best Practice) Relying on a developer to manually call .Dispose() is risky; if an exception is thrown before the method is reached, the resource remains locked. The using statement provides a foolproof wrapper. It acts as a try-finally block under the hood, guaranteeing that Dispose() is executed the exact moment the block ends, even if a critical error occurs inside it.

```
C#
```

```
// The StreamReader (an unmanaged resource) is guaranteed to be disposed here.
```

```
using (StreamReader reader = new StreamReader("data.txt"))
```

```
{
```

```
    Console.WriteLine(reader.ReadToEnd());
```

```
} // Dispose() is implicitly called right at this bracket.
```

3. Finalizers / Destructors (The Safety Net) A Finalizer (written with a tilde, e.g., ~MyClass()) is a special method that the Garbage Collector calls right before it destroys the object in memory. It acts as a backup plan to recover unmanaged resources if the developer completely forgot to use a using statement or call Dispose().

- **The Catch:** You should never rely solely on a Finalizer. The GC runs unpredictably (non-deterministically). If you rely on a Finalizer to close a file, that file might remain locked for seconds or minutes until the GC finally decides to run.

4. The Standard Dispose Pattern For robust enterprise code, Microsoft defines the "Standard Dispose Pattern." This is a specific architectural blueprint where a class implements both IDisposable (for immediate, deterministic cleanup) and a Finalizer (as a failsafe). The pattern includes logic to ensure that if Dispose() is called, the GC is told to skip the Finalizer via GC.SuppressFinalize(), saving CPU cycles.

Ref type passing - self

In C#, reference types (like classes, arrays, and strings) store a memory address pointing to the actual data on the heap, rather than the data itself. How you pass this reference to a method completely changes how memory and object assignment are handled.

The Default Behavior (Passing Reference Types by Value) By default, all parameters in C# are passed "by value." When you pass a reference type, the method receives a **copy of the memory address**, not the object itself.

- **Modifying Properties:** If you change a property inside the method (e.g., user.Name = "Ahmed"), the original object is updated. This happens because the copied memory address still points to the exact same object on the heap.
- **Reassigning the Object:** If you assign a brand new object to the variable inside the method (e.g., user = new User()), the copied address updates to point to the new object. However, the original variable back in the calling method remains completely untouched and continues pointing to the old object.

The ref Keyword (Passing Reference Types by Reference) When you add the ref modifier to a parameter, you are passing a **pointer to the original reference**. The method does not get a copy; it gets direct access to the original variable's memory slot.

- **Reassigning the Object:** If you assign a new object to the parameter inside the method, the original variable in the calling method is instantly overwritten. It will now point to the newly created object, and the old object is abandoned on the heap.

Execution Comparison

Action inside the Method	Default (Passed by Value)	Using ref (Passed by Reference)
obj.Value = 99;	Updates the original object.	Updates the original object.
obj = null;	Original object remains intact.	Original object is set to null.
obj = new Class();	Original object remains intact.	Original object is permanently replaced.

Important Note for the Report: A common misconception is that reference types are automatically passed by reference. They are not. They are references passed by value, which is why the ref keyword is still strictly required if you intend to completely swap out the underlying object in memory.

